ORBIS NEO

GitHub-repo stappen

Inrichting van repository, branchbescherming, Netlify en toegang

Organisatie	**Orbis Neo**
Versie	**1.0**
Datum	**15 oktober 2026**
Auteur	**Vera (Website & Voice) voor Daniel Fokker**
Eigenaar	**Daniel Fokker**
Status	**Concept ter goedkeuring**

Inhoudsopgave

Versiebeheer **3**

Inleiding **4**

Doel van dit document 4

Scope 4

Doelgroep 4

1 Voorwaarden **5**

2 Repository aanmaken **6**

3 Bescherming van main **7**

4 GitHub Actions **8**

5 Netlify koppelen **9**

6 Omgevingsvariabelen en geheimen **10**

7 Inloggen in Decap (GitHub-backend) **11**

8 Toegang voor Claude Code en Vera **12**

9 Controle na inrichting **13**

10 Open punten **14**

Versiebeheer

Versie	Datum	Wijziging	Auteur
1.0	15 oktober 2026	Eerste versie van de stappen voor GitHub en Netlify	Vera (Website & Voice)

Inleiding

Doel van dit document

Dit document beschrijft stap voor stap hoe Daniel de private GitHub-repository voor de nieuwe website aanmaakt, `main` beschermt, Netlify koppelt, deploy previews inschakelt, de benodigde instellingen en geheimen vastlegt en Claude Code en Vera veilig toegang geeft. Na deze stappen kan Claude Code beginnen met pull request 1 uit de startopdracht.

Scope

Binnen scope	Buiten scope
GitHub-organisatie of -account, repository, beveiliging van `main`, CODEOWNERS	De code zelf (zie startopdracht)
Netlify-team, koppeling met GitHub, deploy previews, omgevingsvariabelen, OAuth voor Decap	DNS bij TransIP (zie TransIP DNS)
Toegang voor Claude Code en Vera met minimale rechten	Betaalgegevens en abonnementen invullen

Doelgroep

Daniel Fokker, die de accounts aanmaakt en beheert, en Claude Code en Vera, die moeten weten welke rechten ze hebben.

1 Voorwaarden

Daniel heeft een eigen GitHub-account met tweestapsverificatie en een Netlify-account op een adres van Orbis Neo. Een belangrijk aandachtspunt: beschermde branches, verplichte reviewers en code owners zijn bij GitHub voor **private** repositories alleen beschikbaar met GitHub Pro (persoonlijk account) of GitHub Team (organisatie); met GitHub Free werken ze alleen in publieke repositories (bron: docs.github.com, “GitHub’s plans” en “About rulesets”, gecontroleerd oktober 2026). Omdat de afspraak “niets naar `main` zonder Daniel” op die bescherming steunt, is een van die plannen nodig, of moet Daniel bewust kiezen voor een publieke repository (de site is toch openbaar; er staan geen geheimen in de code). Dit is open punt 1 van dit document.

2 Repository aanmaken

1. Maak bij voorkeur een GitHub-organisatie aan, bijvoorbeeld `orbisneo`, met Daniel als eigenaar. Een organisatie maakt het later eenvoudiger om toegang te geven en in te trekken.
2. Maak de repository `orbisneo-website` aan: zichtbaarheid Private, met een README, `.gitignore` voor Node en zonder licentie.
3. Zet onder Settings, General: standaardbranch `main`; alleen “Allow squash merging” aan; “Automatically delete head branches” aan.
4. Zet onder Settings, Code security: Dependabot alerts en security updates aan, en secret scanning met push protection aan (voor zover het plan dat biedt).

3 Bescherming van main

Maak onder Settings, Rules, Rulesets een branch-ruleset “main beschermd” voor de standaardbranch, met:

Regel	Instelling
Restrict deletions	Aan
Block force pushes	Aan
Require a pull request before merging	Aan, 1 goedkeuring
Require review from Code Owners	Aan
Dismiss stale approvals when new commits are pushed	Aan
Require approval of the most recent reviewable push	Aan, zodat niemand zijn eigen laatste wijziging goedkeurt
Require conversation resolution before merging	Aan
Require status checks to pass	Aan: `ci / build-and-check` en `netlify/deploy-preview` (namen verschijnen na de eerste run)
Bypass list	Leeg, of alleen Daniel voor noodgevallen

Leg in de repository het bestand `.github/CODEOWNERS` vast, zodat elke wijziging Daniels goedkeuring vraagt:

```
# .github/CODEOWNERS
*  @<github-gebruikersnaam-van-daniel>
```

En een sjabloon voor pull requests:

```
<!-- .github/pull_request_template.md -->
## Wat is gewijzigd
## Waarom
## Hoe te testen (preview-URL en stappen)
## Placeholders en open vragen
## Checklist
- [ ] Geen verzonnen feiten, klanten of prijzen
- [ ] Geen em-streepjes
- [ ] Beide talen bijgewerkt
```

4 GitHub Actions

De workflow controleert elke pull request. Netlify bouwt en publiceert zelf; Actions hoeft daarvoor geen geheimen te hebben.

```
# .github/workflows/ci.yml
name: ci
on:
  pull_request:
  push:
    branches: [main]
permissions:
  contents: read
jobs:
  build-and-check:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version-file: .nvmrc
          cache: npm
      - run: npm ci
      - run: npx astro check
      - run: npm run build
        env:
          CONTEXT: deploy-preview
          PUBLIC_NOINDEX: "true"
      # npm run build draait ook check-content en check-redirects
      - name: Linkcheck
        uses: lycheeverse/lychee-action@v2
        with:
          args: --offline --no-progress "dist/**/*.html"
      - name: Lokale server voor toegankelijkheidstest
        run: npx --yes serve dist -l 4321 &
      - name: Toegankelijkheid (axe)
        run: npx --yes wait-on http://localhost:4321 && npx --yes @axe-core/cli http://localhost:4321/
http://localhost:4321/welkom/ http://localhost:4321/contact/ http://localhost:4321/kennismaken/ --exit
```

Lighthouse wordt door Vera op de deploy preview gedraaid (zie werkafspraak); een automatische Lighthouse-run op de preview kan later met de Netlify Lighthouse-plugin worden toegevoegd.

5 Netlify koppelen

1. Maak in Netlify een team aan op naam van Orbis Neo.
2. Kies “Add new project”, “Import an existing project”, GitHub, en geef de Netlify GitHub App alleen toegang tot `orbisneo-website` (niet tot alle repositories).
3. Buildinstellingen worden uit `netlify.toml` gelezen: command `npm run build`, publish `dist`.
4. Geef de site een herkenbare naam, bijvoorbeeld `orbisneo-website`; het Netlify-adres wordt dan `orbisneo-website.netlify.app`. Die naam is nodig voor de CNAME-records in “TransIP DNS”.
5. Controleer onder Project configuration, Build and deploy, Branches and deploy contexts: production branch `main`; Deploy Previews voor alle pull requests naar `main` aan; branch deploys uit.
6. Zet onder Deploy notifications de GitHub-commitstatus en de pull-request-reactie met de preview-link aan.
7. Zet onder Forms de formulierdetectie aan en stel bij Form notifications het ontvangstadres in (open punt 4 van het productdocument).

6 Omgevingsvariabelen en geheimen

Naam	Waar	Waarde	Geheim
`PUBLIC_NOINDEX`	Netlify, alle contexten	`true` tot de lancering; productie daarna `false`	Nee
`PUBLIC_SITE_ENV`	`netlify.toml` per context	`production` of `preview`	Nee
`PUBLIC_UMAMI_WEBSITE_ID`	Netlify, productie	Website-id uit Umami	Nee (staat in de HTML)
`LANG_USE_GEO`	Netlify, alle contexten	`false` (open punt 2 productdocument)	Nee
GitHub OAuth client ID en secret	Netlify, Access and security, OAuth	Uit de GitHub OAuth App (zie hieronder)	Ja, alleen in Netlify

Er staan geen geheimen in de repository. GitHub Actions heeft voor deze opzet geen eigen geheimen nodig.

7 Inloggen in Decap (GitHub-backend)

1. Maak in GitHub onder Settings, Developer settings, OAuth Apps een nieuwe OAuth App “Orbis Neo CMS” met homepage `https://www.orbisneoconsulting.com` en callback-URL `https://api.netlify.com/auth/done`.
2. Voer in Netlify onder Project configuration, Access and security, OAuth bij “Install provider” GitHub de client ID en het client secret in.
3. In `public/admin/config.yml` staat `backend: name: github`, `repo: orbisneo/orbisneo-website` en `branch: main`.
4. Daniel en Vera loggen op `/admin/` in met hun eigen GitHub-account; ze hebben schrijfrechten op de repository, en door de ruleset komt alles als pull request binnen.

8 Toegang voor Claude Code en Vera

Wie	Hoe	Rechten	Niet toegestaan
Claude Code	Een fine-grained personal access token van een apart GitHub-account (bijvoorbeeld `orbisneo-builder`), alleen voor `orbisneo-website`, of de officiële Claude GitHub App alleen voor deze repository	Contents read and write, Pull requests read and write, Metadata read; vervaldatum maximaal 90 dagen	Admin, bypass van de ruleset, toegang tot andere repositories
Vera	Eigen GitHub-account of GitHub App, rol Write op deze repository	Branches en pull requests maken, reageren	Samenvoegen naar `main` zonder Daniels goedkeuring, rulesets wijzigen
Daniel	Eigenaar	Admin, enige code owner	n.v.t.

Tokens worden niet in de repository of in chats geplakt, maar alleen in de omgeving waar Claude Code draait. Bij beëindiging van de opdracht trekt Daniel het token in of verwijdert hij het account uit de organisatie.

9 Controle na inrichting

1. Claude Code opent een test-pull request; de Netlify-reactie met preview-link verschijnt.
2. De ruleset blokkeert samenvoegen zonder Daniels goedkeuring en zonder groene controles.
3. Een directe push naar `main` door Claude Code wordt geweigerd.
4. Daniel kan op de preview `/admin/` openen en inloggen.

10 Open punten

#	Vraag	Voorstel
1	Private repository met GitHub Pro of Team, of publieke repository met GitHub Free?	Private met Team (organisatie) of Pro; publiek kan ook, omdat er geen geheimen in staan
2	Naam van de GitHub-organisatie en de repository	`orbisneo` en `orbisneo-website`
3	Claude Code via apart account met token of via de Claude GitHub App?	Apart account met fine-grained token, 90 dagen